Lab 2: High-Performance Monte Carlo – Intro to OpenMP

Course: Parallel Computing
Duration: 80 Minutes
Environment: VS Code (C/C++)

1. The Scenario

You are a **Software Optimization Specialist** for a high-performance computing center. A research team has provided a legacy serial program that performs a **Monte Carlo Simulation** to calculate the mathematical constant π.

The current single-threaded implementation is too slow for their dataset of **100 million samples**. They have asked you to rewrite the core algorithm using **OpenMP** to exploit the full parallelism of their multi-core servers.

The Challenge:

This is an "embarrassingly parallel" problem (perfectly independent iterations), yet naive parallelization often fails due to hidden resource contention in the C standard library. Your job is to identify this bottleneck and implement a thread-safe, high-speed solution.

The Algorithm:

We estimate π by throwing random "darts" at a 1 x 1 square with an inscribed circle.

$$\pi \approx 4.0 \times \frac{Hits\ Inside\ Circle}{Total\ Darts\ Thrown}$$

A sample at (x, y) is inside the circle if $x^2 + y^2 \leq 1.0$.

2. Lab Objectives

1. **Decompose** a computational workload for parallel execution.
2. **Identify and fix** Race Conditions using OpenMP Reductions.
3. **Analyze** the "False Sharing" performance trap caused by non-thread-safe functions (rand()).
4. **Benchmark** speedup metrics across 1, 2, 4, and 8 threads.

Phase 1: Algorithm Analysis & Strategy

Deliverable: Create a text file named strategy.txt.

Do not write code yet. Analyze the algorithm and plan your implementation by answering the following:

1. **Decomposition Strategy:** The loop runs 100,000,000 times. Is the computational cost per iteration constant or variable? Based on this, which OpenMP schedule (static or dynamic) allows for the lowest scheduling overhead?
2. **Memory Model:**

- Which variables must be private (thread-local stack) to prevent data corruption?
- Which variables must be shared (heap/global)

3. **Concurrency Control:** If multiple threads identify a "hit" simultaneously, they will race to write to the counter. Which OpenMP construct is best for aggregating a simple sum: critical, atomic, or reduction? Justify your choice based on performance.

Phase 2: Baseline Profiling (Serial)

Deliverable: pi_serial.c

Create a standard serial implementation to establish a **Performance Baseline**.

1. **Write the Code:**
   - Include <stdio.h>, <stdlib.h>, and <omp.h> (for the timer).
   - Set long num_steps = 100000000; (100 million).
   - Implement the logic:
     - Start Timer: double start = omp_get_wtime();
     - Loop num_steps times.
     - Generate random x and y using (double)rand() / RAND_MAX.
     - Compute $x^2 + y^2$. If ≤ 1.0, increment hits.
     - End Timer: double end = omp_get_wtime();
   - Print the calculated π and the **Execution Time**.
2. **Compile & Run:**
   - Command: gcc -fopenmp pi_serial.c -o serial
   - Run: ./serial
3. **Record:** Write the time in strategy.txt as your **Base Time (T_{serial})**.

Phase 3: Naive Parallelization (The Trap)

Deliverable: pi_naive.c

Apply OpenMP pragmas to the code without modifying the underlying helper functions.

1. **Create the File:** Copy pi_serial.c to pi_naive.c.
2. **Add OpenMP:**
   - Decorate the main loop with #pragma omp parallel for.
   - Apply the scoping (private/shared) and reduction clauses you decided on in Phase 1.

- **Constraint:** Do not change the random number generation (rand()) yet.

3. **Run with 4 Threads:**
    - Set threads: export OMP_NUM_THREADS=4 (Linux/Mac) or env:OMP_NUM_THREADS='4' (Windows).
    - Compile and Run.
4. **Analyze:**
    - Compare the time to T_{serial}.
    - **Observation:** You will likely see **zero speedup** (or a slowdown). In strategy.txt, explain why calling rand() (which maintains a hidden global state) inside a parallel region serializes the execution.

Phase 4: Thread-Safe Optimization

Deliverable: pi_optimized.c

To achieve linear speedup, you must eliminate the shared state dependency. You will implement a **Thread-Local Storage (TLS)** approach using rand_r().

1. **Refactor the Code:**
    - Copy your naive code to pi_optimized.c.
    - Inside the parallel region (but *before* the loop), declare a **private seed** unique to that thread:

```
// XOR current time with thread ID to ensure unique sequences
unsigned int seed = time(NULL) ^ omp_get_thread_num();
```

    - Update the loop to use rand_r(&seed), which modifies only the local seed:

```
x = (double)rand_r(&seed) / RAND_MAX;
y = (double)rand_r(&seed) / RAND_MAX;
```

    - *Note:* Declaring seed inside the parallel block automatically makes it private.
2. **Validation:**
    - Compile: gcc -fopenmp pi_optimized.c -o optimized
    - Run with 4 threads. You should now observe significant performance gains.

Phase 5: Scaling Analysis

Deliverable: Finalize strategy.txt.

Run your pi_optimized program with 1, 2, 4, and 8 threads. Calculate the **Speedup** for each configuration.

Speedup(N) = $T_{serial}/T_{parallel(N)}$

1. Fill out this table in your text file:

Threads	Execution Time (s)	Speedup	Ideal Speedup
1	*(from Phase 2)*	1.0x	1.0x
2			2.0x
4			4.0x
8			8.0x

2. Discussion:

Did you achieve perfect linear speedup (e.g., 4.0x on 4 threads)? If not, identify one factor contributing to the overhead (e.g., thread creation cost, memory bandwidth saturation, or false sharing).

Appendix: VS Code Terminal Cheat Sheet

- **Open Terminal:** Ctrl + ~ (tilde)
- Compile (GCC):

gcc -fopenmp filename.c -o outputname

- Run (Windows):

./outputname or .\outputname.exe

- Set Threads (Linux/Git Bash):

export OMP_NUM_THREADS=4

- Set Threads (Windows PowerShell):

env:OMP_NUM_THREADS='4'